

---

# Java Programming

## CO2-AT2

**Name: Preethika M**

**Reg No: 192421084**

---

**Compare ArrayList, HashSet, HashMap and Hashtable by data model, ordering, duplicates, null support, access method, synchronisation and typical complexity. Recommend one for four different application requirements. [Evaluate]**

### **Comparison of ArrayList, HashSet, HashMap and Hashtable**

Java provides different collection classes for storing and managing data. **ArrayList, HashSet, HashMap, and Hashtable** are commonly used collections, but they differ in the way they store data, handle duplicates and null values, maintain ordering, and provide access to elements.

Choosing the correct collection depends on the requirements of the application.

#### **1. ArrayList**

An **ArrayList** is a dynamic array that stores elements in a sequential manner. It automatically increases its capacity when more elements are added.

#### **Characteristics**

- **Data Model:** Stores individual elements in a list.
- **Ordering:** Maintains insertion order.
- **Duplicates:** Duplicate elements are allowed.
- **Null Support:** Allows multiple null values.
- **Access Method:** Elements can be accessed using their index.
- **Synchronization:** Not synchronized by default.
- **Access Complexity:**  $O(1)$  for accessing an element by index.
- **Search Complexity:**  $O(n)$ .

- **Insertion at End:**  $O(1)$  amortized.
- **Middle Insertion/Deletion:**  $O(n)$ .

## Example

```
ArrayList<String> names = new ArrayList<>();
```

```
names.add("Ram");  
names.add("John");  
names.add("Ram");
```

Result:

[Ram, John, Ram]

Here, the duplicate value "Ram" is allowed.

---

## 2. HashSet

A **HashSet** stores a collection of unique elements. It is useful when duplicate values should not be allowed.

### Characteristics

- **Data Model:** Stores unique individual elements.
- **Ordering:** Does not guarantee insertion order.
- **Duplicates:** Not allowed.
- **Null Support:** Allows one null element.
- **Access Method:** Elements are mainly accessed using iteration or membership operations such as `contains()`.
- **Synchronization:** Not synchronized by default.
- **Add Complexity:**  $O(1)$  average.
- **Search Complexity:**  $O(1)$  average.
- **Remove Complexity:**  $O(1)$  average.

## Example

```
HashSet<String> cities = new HashSet<>();
```

```
cities.add("Chennai");  
cities.add("Mumbai");
```

```
cities.add("Chennai");
```

The second "Chennai" is not added because HashSet does not allow duplicate elements.

Conceptually, the set contains:

Chennai, Mumbai

The display order is not guaranteed.

---

### 3. HashMap

A **HashMap** stores information in the form of **key-value pairs**.

For example:

Student ID → Student Name

101 → Ravi

102 → Priya

Each key must be unique, while values can be duplicated.

#### Characteristics

- **Data Model:** Key-value pairs.
- **Ordering:** Does not guarantee insertion order.
- **Duplicates:** Duplicate keys are not allowed, but duplicate values are allowed.
- **Null Support:** Allows one null key and multiple null values.
- **Access Method:** Values are accessed using their keys.
- **Synchronization:** Not synchronized by default.
- **Insertion Complexity:**  $O(1)$  average.
- **Search Complexity:**  $O(1)$  average.
- **Deletion Complexity:**  $O(1)$  average.

#### Example

```
HashMap<Integer, String> students = new HashMap<>();
```

```
students.put(101, "Ravi");
```

```
students.put(102, "Priya");
```

```
students.put(103, "Ravi");
```

Here "Ravi" can appear more than once because duplicate **values** are allowed.

---

## 4. Hashtable

A **Hashtable** also stores information as key-value pairs and is similar to HashMap. However, Hashtable is synchronized.

### Characteristics

- **Data Model:** Key-value pairs.
- **Ordering:** Does not guarantee insertion order.
- **Duplicates:** Duplicate keys are not allowed, but duplicate values are allowed.
- **Null Support:** Does not allow null keys or null values.
- **Access Method:** Values are accessed using their keys.
- **Synchronization:** Synchronized and thread-safe for individual operations.
- **Insertion Complexity:**  $O(1)$  average.
- **Search Complexity:**  $O(1)$  average.
- **Deletion Complexity:**  $O(1)$  average.

Because Hashtable uses synchronization, it can have additional performance overhead compared with HashMap.

## Comparison Table

| Feature         | ArrayList                         | HashSet                             | HashMap                               | Hashtable                             |
|-----------------|-----------------------------------|-------------------------------------|---------------------------------------|---------------------------------------|
| Data Model      | List of elements                  | Set of unique elements              | Key-value pairs                       | Key-value pairs                       |
| Ordering        | Maintains insertion order         | No guaranteed order                 | No guaranteed order                   | No guaranteed order                   |
| Duplicates      | Allowed                           | Not allowed                         | Duplicate values allowed, keys unique | Duplicate values allowed, keys unique |
| Null Support    | Multiple nulls                    | One null                            | One null key and multiple null values | No null keys or values                |
| Access Method   | By index                          | <code>contains()</code> / iteration | By key                                | By key                                |
| Synchronization | No                                | No                                  | No                                    | Yes                                   |
| Access/Search   | O(1) index access;<br>O(n) search | O(1) average                        | O(1) average                          | O(1) average                          |
| Insertion       | O(1) amortized at end             | O(1) average                        | O(1) average                          | O(1) average                          |
| Deletion        | O(n) from middle                  | O(1) average                        | O(1) average                          | O(1) average                          |

The O(1) operations of HashSet, HashMap, and Hashtable are average-case complexities. Poor hash distribution or collisions can make operations slower.

## Recommendations for Different Application Requirements

### Requirement 1: Maintaining a Playlist

Suppose a music application needs to maintain a playlist where songs should appear in the same order in which the user added them. The same song may also be added multiple times.

#### Recommended Collection: ArrayList

**Reason:** ArrayList maintains insertion order, allows duplicate elements, and provides fast access using an index.

Example:

[Song A, Song B, Song A, Song C]

Therefore, ArrayList is suitable when **ordering and index-based access are important**.

---

### Requirement 2: Storing Unique Usernames

Suppose a social media application needs to store usernames and should prevent duplicate usernames.

#### Recommended Collection: HashSet

**Reason:** HashSet stores only unique elements and provides approximately  $O(1)$  average performance for checking whether an element already exists.

For example:

preethi\_01  
john\_25  
alex\_10

If **preethi\_01** is added again, HashSet will not store another copy.

Therefore, HashSet is suitable when **uniqueness is the main requirement**.

---

## Requirement 3: Student ID and Student Name Lookup

Suppose a college application stores student information using:

101 → Ravi

102 → Priya

103 → John

### Recommended Collection: HashMap

**Reason:** HashMap stores key-value pairs and allows fast retrieval using a unique key.

For example:

```
students.get(102);
```

can retrieve the student associated with ID 102.

HashMap provides approximately **O(1) average lookup performance**.

Therefore, HashMap is suitable when **fast key-based lookup is required and synchronization is not necessary**.

---

## Requirement 4: Shared Key-Value Data Accessed by Multiple Threads

Suppose multiple threads in an application need to access shared key-value data and the requirement specifically calls for a synchronized legacy map.

### Recommended Collection: Hashtable

**Reason:** Hashtable stores key-value pairs and its methods are synchronized. This means individual operations can be safely accessed by multiple threads without external synchronization.

However, Hashtable generally has more synchronization overhead than HashMap and does not allow null keys or values.

Therefore, Hashtable can be considered when **synchronized key-value access is explicitly required**.

For modern Java applications, classes such as `ConcurrentHashMap` are generally preferred for highly concurrent workloads, but among the four collections compared here, **Hashtable is the synchronized option**.

---

## Final Evaluation

The choice of collection depends on the application's requirements.

- **ArrayList** is best when ordering, duplicates, and index-based access are important.
- **HashSet** is best when only unique elements should be stored and fast membership checking is required.
- **HashMap** is best when information must be stored as key-value pairs with fast lookup using keys.
- **Hashtable** provides synchronized key-value operations but has additional synchronization overhead and does not support null keys or values.

Therefore, there is no single collection that is best for every application. The appropriate collection should be selected based on **data representation, ordering, duplicate handling, null requirements, access pattern, synchronization needs, and expected performance**.